

EECS 3311
Project Deliverable 3
Group 16
Omar Fakousa,
Adan Sheikh,
Declan Palmateer,
Salik Karim

Github repo link: https://github.com/Decpalm/EECS_3311_Reservation_System

Task 2. The output of Randoop

The automated testing phase involved the execution of Randoop against the non-GUI classes of the reservation system, as dictated by the [randoop-classlist.txt](#) configuration. This scope encompassed components within the [model](#), [state](#), [strategy](#), and [factory](#) packages. Randoop generated 19,700 JUnit test cases across 20 regression test files, all of which achieved passing results with 0 errors and 0 failures. The output comprised two distinct categories. Regression tests represent valid sequences of method invocations that produced anticipated system behavior, verifying that the implementation does not regress under repeated API calls. Conversely, error tests identify sequences causing exceptions or unexpected states, documenting the boundary between valid and invalid usage discovered through automated random method sequencing.

The cumulative instruction coverage achieved by the Randoop test suite was measured at 91.3% across the test directory. Optimal coverage of 100% was attained for several classes, including [Student.java](#), [User.java](#), and all four concrete strategy implementations: [FacultyPricingStrategy](#), [GuestPricingStrategy](#), [ResearcherPricingStrategy](#), and [StudentPricingStrategy](#). Additionally, four state classes—[AvailableState](#), [CancelledState](#), [DisabledState](#), and [MaintenanceState](#)—reached full coverage. These results underscore Randoop's efficacy when targeting stateless components with accessible constructors and minimal external dependencies.

Conversely, Randoop exhibited significant limitations when analyzing complex domain classes. Coverage for [Reservation.java](#) was restricted to 12.9% because the class necessitates the concurrent instantiation of three dependent objects—[User](#), [Equipment](#), and [PricingStrategy](#)—a configuration that random sequencing rarely satisfies before execution timeouts. [Payment.java](#) remained at 0% coverage as the tool failed to identify valid invocation paths. Packages involving [gui](#), [app](#), [command](#), and [observer](#) similarly recorded 0% coverage, consistent with Randoop's inability to drive event-based sequences or GUI interactions. [Equipment.java](#) reached 49.6% coverage; however, methods such as [isAvailable\(\)](#) and [isModifyAvailable\(\)](#) were excluded because they utilize [CSVDataStore.getInstance\(\)](#), which requires a local CSV directory. These constraints illustrate the fundamental trade-off of automated random testing: while it offers effortless broad coverage for simple logic, it struggles to synthesize the complex object graphs required by sophisticated domain models.

Task 3. AI-Generated Test Cases

The AI assistant used for this deliverable was Claude.

Prompts and AI Responses

The interaction began with a structured role-assignment prompt in which Claude was instructed to act as a senior Java testing engineer and to analyse the deliverable requirements, identify testable classes, generate JUnit 5 test cases, and produce a report section. Claude responded by extracting a precise checklist of deliverable requirements and requesting the source code before proceeding. A second prompt provided the GitHub repository link along with direct pastes of eight source files: [Reservation.java](#), [Equipment.java](#), [User.java](#), [AccountFactory.java](#), [EquipmentFactory.java](#), [ResearcherPricingStrategy.java](#), [ReservationSystem.java](#), and [CSVDataStore.java](#). Claude responded by identifying six testable non-GUI classes and generating 131 test cases across six test files. A third refinement prompt instructed Claude to exclude ReservationSystem and CSVDataStore from unit test scope, since both are Singletons whose constructors immediately perform CSV file I/O, making them unsuitable for isolated unit testing without a mocking framework. Claude accepted this constraint and removed those classes from scope, also correctly excluding [Equipment.isAvailable\(\)](#) and [Equipment.isModifyAvailable\(\)](#) for the same reason.

Acceptance, Modification, and Rejection of Generated Tests

The majority of generated tests were accepted without modification. These included all tests for [AccountFactory](#), [EquipmentFactory](#), [User](#), [ResearcherPricingStrategy](#), and [ReservationTest](#). Two categories of tests were modified. First, the SensorUpdate constructor calls in EquipmentTest were corrected: Claude assumed a single-argument constructor ([new SensorUpdate\("AVAILABLE"\)](#)), but the actual class requires two arguments — an operational status string and a message string — so all five affected lines were updated to new [SensorUpdate\("AVAILABLE", "Sensor back online"\)](#) and equivalents. Second, three observer tests required assertion corrections after runtime execution revealed that [equipment.disable\(\)](#) triggers [notifyObservers\(\)](#) twice rather than once. This occurred because [disable\(\)](#) calls [setState\(\)](#) internally, which already invokes [notifyObservers\(\)](#), and then calls [notifyObservers\(\)](#) again explicitly at the end of the method. Claude had assumed a single notification per state change. The assertions in [attach_observer_isNotifiedOnStateChange](#), [attach_observer_receivesCorrectEquipmentReference](#), and [multipleObservers_allNotified](#) were corrected from [assertEquals\(1, obs.callCount\)](#) to [assertEquals\(2, obs.callCount\)](#). No tests were outright rejected, though tests for ReservationSystem and CSVDataStore were scoped out as described above.

Prompt Refinement

The primary issue with the initial prompt was insufficient scoping — it did not instruct Claude to exclude classes with file I/O or Singleton dependencies. Without the refinement prompt, Claude would have generated tests for ReservationSystem and CSVDataStore that would have thrown FileNotFoundException at runtime because the data/ CSV directory is not present in the test environment. Restricting the scope to pure model, factory, and strategy classes resolved this issue entirely.

Conflicts Between AI Assumptions and Actual Class Behaviour

Two conflicts arose between Claude's assumptions and the actual implementation. The first was the SensorUpdate constructor signature described above. The second was the double-notification behaviour in Equipment. Claude's generated tests assumed that each call to [disable\(\)](#), [enable\(\)](#), or [markMaintenance\(\)](#) would trigger exactly one observer notification, which is the standard Observer pattern expectation. In reality, the implementation calls [notifyObservers\(\)](#) both inside [setState\(\)](#) and again at the end of each transition method, producing two notifications per call. This conflict was identified by running the test suite and reading the failure trace, which reported expected: <1> but was: <2>. The resolution was to update the assertions to match the actual observed behaviour and to document the double-notification as a known implementation characteristic.

Verification of Correctness

Correctness of the AI-generated tests was verified by compiling and running the full suite in Eclipse using JUnit 5. The initial run produced 134 passes and 2 failures, both in EquipmentTest, due to the observer count mismatch described above. After correcting the three affected assertions, the suite reached 136/136 passing with 0 errors. Code coverage was measured using EcJemma and achieved 95.6% instruction coverage across the ai_generated test suite. Classes with

100% coverage included [User.java](#), [Guest.java](#), [ResearcherPricingStrategy.java](#), and [StudentPricingStrategy.java](#). [Reservation.java](#) achieved 96.1% and [AccountFactory.java](#) achieved 95.4%. Lower coverage on [Equipment.java](#) (49.9%) and [Payment.java](#) (31.8%) is attributable to the intentional exclusion of methods that depend on CSVDataStore, which is appropriate for a unit test suite. The overall coverage of 95.6% substantially exceeds the 80% threshold required by the deliverable.

Task 4. Comparison between the approaches

Code Coverage: The manually written tests achieved the highest overall coverage at 88.5%. This is because the manual tests were written with full understanding of the system architecture and intentionally exercised every package, which the other two methods could not complete. The cumulative instruction coverage achieved by the AI-generated test suite was measured at 95.6% across the specifically targeted classes. Randoop's weakness on model is attributable to Reservation.java achieving only 12.9% coverage, since Randoop's random method sequencing rarely assembled the required combination of User, Equipment, and PricingStrategy objects needed to construct a Reservation. In summary, manual testing covered the broadest scope of the codebase, the AI-generated suite achieved the deepest targeted coverage within its defined scope, and Randoop performed well on simple stateless classes but fell short on complex domain objects requiring multi-step object construction.

Mutation Score: The mutation score analysis indicates that the manual test suite was the most effective at detecting injected faults. This is because the manual tests were intentionally designed to validate core system behavior, edge cases, and failure conditions, such as incorrect permissions, invalid inputs, and state transitions. As a result, these tests contain stronger and more meaningful assertions, making them more likely to fail when faults are introduced, thereby killing more mutants. The automatically generated test suites, particularly those produced by Randoop, primarily focus on achieving higher code coverage and often verify only basic execution or non-crashing behavior. While these tests may execute a large portion of the codebase, they are less effective at detecting subtle logical errors, resulting in a lower mutation score. The AI-generated tests fall in between, providing some targeted validation but still lacking the depth and intent of the manual tests. Overall, the manual test suite achieved the highest mutation score due to its focus on correctness and comprehensive validation of system behavior.

Readability: Manually written tests are the easiest to follow and maintain. They are grouped in packages that match production code (model, strategy, persistence), use JUnit 5 and descriptive method names (eg. emptyIdOrCert_deactivatesUser, saveReservation_findReservationById), and keep each test focused on one behaviour. A reader can tell what is being checked without digging through setup. The main downside is duplication if older suites are kept alongside them, which can confuse maintenance. Randoop-generated tests score lowest on readability. Methods are named test0001, test0002, and so on, so names do not describe intent. Files are very large, mix many classes, and include boilerplate. They document what the tool executes, not what requirement is being validated. Fixing a failing Randoop test often means reading the whole sequence rather than a clearly named scenario. AI-assisted tests sit in the middle. They use clear names, section comments, and class-level Javadoc explaining strategy (eg. testing User via subclasses). They read like hand-written tests but can be verbose or uneven in style across files. Overall, manual tests are best structured for humans, AI-assisted are readable if curated, and Randoop is useful as a machine-generated safety net but is hard to navigate and costly to maintain without renaming or refactoring.

Usefulness: The manual tests meaningfully cover all the classes, testing proper functionality and edge cases, like reservation overlap detection and the full state machine lifecycle. The usefulness of AI generated tests is limited mainly due to them not covering as much of the project, so many areas

of the code end up untested. For the parts of the code it does cover, though it is very thorough with what it does cover, like with its numeric pricing assertions. The randoop tests are the least useful. While they do cover a large amount of the code, the test cases don't catch real faults, as they are generated by recording current behaviour, as opposed to specifying correct behaviour, as is done in the AI and manual tests. Overall, the manual test cases are the most useful, and the randoop generated tests are the least useful.